Course Discussions

Solution: Number of Provinces (medium)

Problem Statement

Imagine a country with several cities. Some cities have direct roads connecting them, while others might be connected through a sequence of intermediary cities. Using a matrix representation, if `matrix[i][j]` holds the value `1`, it indicates that city `i` is directly linked to city `j` and vice versa. If it holds `0`, then there's no direct link between them.

Determine the number of separate city clusters (or provinces).

A **province** is defined as a collection of cities that can be reached from each other either directly or through other cities in the same province.

Examples

1. - **Input:**

     ```
     [[1,1,0],[1,1,0],[0,0,1]]
     ```

   - **Expected Output:**
     `2`
   - **Justification:**
     There are two provinces: cities 0 and 1 form one province, and city 2 forms its own province.
2. - **Input:**

     ```
     [[1,0,0,1],[0,1,1,0],[0,1,1,0],[1,0,0,1]]
     ```

   - **Expected Output:**
     `2`
   - **Justification:**
     There are two provinces: cities 0 and 3 are interconnected forming one province, and cities 1 and 2 form another.
3. - **Input:**

     ```
     [[1,0,0],[0,1,0],[0,0,1]]
     ```

   - **Expected Output:**
     `3`
   - **Justification:**
     Each city stands alone and is not connected to any other city. Thus, we have three provinces.

Solution

At a high level, the problem of identifying provinces in the given matrix can be visualized as detecting connected components in an undirected graph. Every city represents a node, and a direct connection between two cities is an edge. The number of separate, interconnected clusters in this graph is essentially the number of provinces. To navigate this graph and identify these clusters, we employ the Depth First Search (DFS) technique, marking visited nodes (cities) along the way.

1. **Initialization**: Start with a `visited` array, initialized with all values set to `false`. This will help in keeping track of cities that have been processed.
2. **DFS Function**: This recursive function allows us to traverse the matrix. When an unvisited city is found, we recursively visit all other cities accessible from it, marking them as visited. All cities traversed in a single DFS invocation belong to the same province.
3. **Counting Provinces**: Every unique invocation of the DFS function on an unvisited city, from the main function, signifies the discovery of a new province. Therefore, for each such invocation, we increment our province count.
4. **Completion**: After every city has been visited, our province counter will hold the total number of provinces in the country.

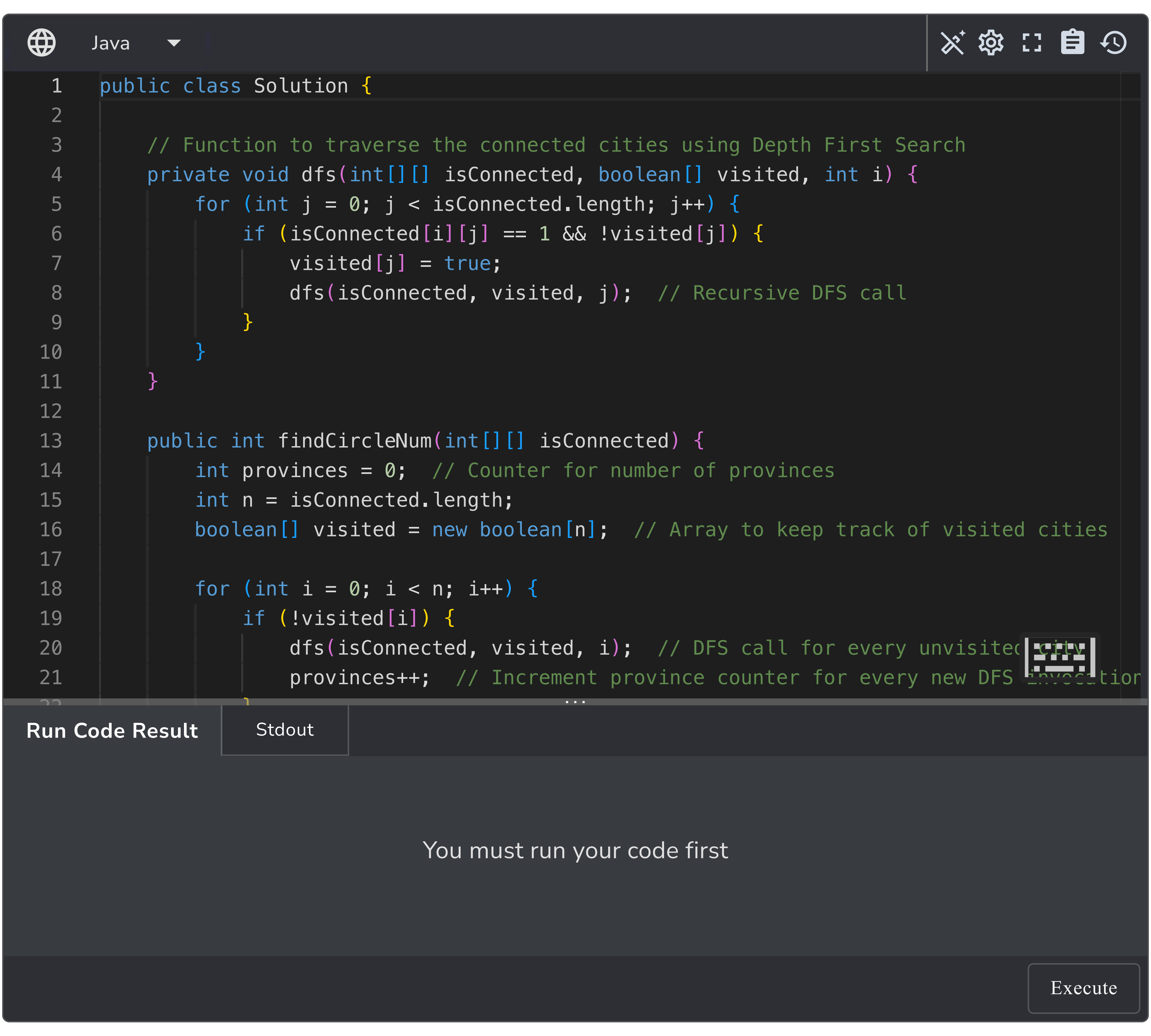
Algorithm Walkthrough: Using the input `[[1,1,0],[1,1,0],[0,0,1]]`:

- Begin with an unvisited `visited` array: `[false, false, false]`.
- Start from city 0:
  - Visit city 0 and city 1 since they are connected. Update `visited` to `[true, true, false]`.

- City 2 remains unvisited, but it isn't connected to any other unvisited city. So, it forms its own province.
- We initiated the DFS twice: once for the province comprising cities {0,1} and once for the solitary city 2.
- Our answer becomes `2`.

Code

Here is the code for this algorithm:



```java
public class Solution {

    // Function to traverse the connected cities using Depth First Search
    private void dfs(int[][] isConnected, boolean[] visited, int i) {
        for (int j = 0; j < isConnected.length; j++) {
            if (isConnected[i][j] == 1 && !visited[j]) {
                visited[j] = true;
                dfs(isConnected, visited, j);  // Recursive DFS call
            }
        }
    }

    public int findCircleNum(int[][] isConnected) {
        int provinces = 0;  // Counter for number of provinces
        int n = isConnected.length;
        boolean[] visited = new boolean[n];  // Array to keep track of visited cities

        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                dfs(isConnected, visited, i);  // DFS call for every unvisited city
                provinces++;  // Increment province counter for every new DFS invocatio
```

Time Complexity

- **Depth First Search (DFS)**: For a given node, the DFS will explore all of its neighbors. In the worst case, we may end up visiting all nodes in the graph starting from a single node. Hence, the DFS complexity is (O(n)), where (n) is the number of nodes.

- **Overall Time Complexity**: For each node, we might call DFS once (if that node is not visited before). Thus, the overall time complexity is (O(n^2)), with the DFS call being nested inside a loop that iterates over all nodes. In dense graphs where each node is connected to every other node, we will reach this upper bound.

Space Complexity

- **Visited Array**: This is an array of size (n) (the number of nodes), so its space requirement is (O(n)).
- **Recursive Call Stack**: In the worst case, if all cities are connected in a linear manner (like a linked list), the maximum depth of recursive DFS calls will be (n). Hence, the call stack will take (O(n)) space.
- **Overall Space Complexity**: The dominant space-consuming factors are the `visited` array and the recursive call stack. Hence, the space complexity is (O(n)).

In summary:

- Time Complexity: (O(n^2))
- Space Complexity: (O(n))

This algorithm is efficient because once a city is visited, it won't be visited again, ensuring we don't do redundant work. Moreover, using DFS allows us to deeply traverse through connected cities, marking entire provinces in one go. This approach optimizes our search and helps in reducing the number of unnecessary computations.

← Previous
Problem 2: Number Of Provinces (Medium)
Next →
Problem 3: Minimum Number Of Vertices To Reach All Nodes(Medium)
☐ Mark as Completed